

SkillWatermark: An Embedded Skill Watermark of Progressive Privacy Inference via Benign Prompts

Yu Li^{1,2}, Liqi Zhuang^{1,2}, Dong Wei^{1,2}, Jiwen Luo³, Hang Zhang¹

Meng Zhang^{1,2}, Xiaona Li¹, Weiqing Huang^{1,2}

Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

CASIC Research Institute of Intelligent Decision Engineering, China.

Abstract—Skills for large language model (LLM) agents have been widely deployed across diverse application domains. However, we observe that these skills generate specific traffic patterns during execution. In this paper, we design a pipeline that generates specific traffic patterns by inserting carefully designed skill descriptions, which we term skill watermarks, so that a passive network attacker can establish a covert channel to encode private information within observable traffic across multiple conversation turns. Specifically, we insert prompt constraint terms, referred to as watermarks, into the original skill descriptions and embed them within multi-turn conversations. The key information in the user’s original prompt is thereby triggered by these watermarks, producing clearly observable encodings in the traffic. The adversary need only decode the traffic patterns to recover the encoded information. In particular, our modifications are benign in the sense that they do not directly exfiltrate any private data and do not execute any malicious instructions. Extensive experiments demonstrate that our watermarks produce highly consistent and distinguishable traffic patterns, and that the transformed skills pass existing LLM-based security auditing tools. This study highlights that generating specific traffic patterns can be exploited as a novel attack surface and offers critical insights for future security hardening.

Index Terms—skill, large language model, watermark

I. INTRODUCTION

Large language model (LLM)-based agents are autonomous systems equipped with perception, reasoning, and execution capabilities that are experiencing rapid growth in the consumer market. Widely adopted and representative agents include Claude Code [1], Cursor [2], and Codex [3]. These LLM agents are being rapidly adopted across industrial and consumer domains including healthcare [4], finance [5], education [6] and software engineering [7]. Agents enhance their capabilities in different domains by installing specialized skills, which are extensible modular toolkits designed for specific tasks. Currently, agent skills ecosystem is experiencing hypergrowth [8].

However, recent studies indicate that the privacy threats introduced by large-scale agent deployment cannot be overlooked. Prior work has shown that private information can be acquired through passive observation of network traffic, even when the user sends no information to any third party during agent interactions. For instance, network traffic observation has been used to conduct behavior inference [9] and profile users [10]. Nevertheless, these works typically rely on pre-built datasets, which means they must collect traffic generated

by specific prompt instructions in advance. Once a user issues a prompt outside the dataset, the attacker treats this as an open-world problem [11], [12] and will ignore such prompts. In particular, in traffic analysis methods that target the private content within prompts, the prompt content is user-dependent and highly variable. This is fundamentally different from traffic analysis targeting apps [12] or websites [13], which typically exhibit nearly fixed traffic patterns.

To fundamentally address the inherent limitations of the established methodology based on data collection, model training, and classification, we introduce a watermark, a specific skill description fragment, into existing skills to generate a short, fixed traffic pattern. When the agent detects a particular keyword in the user’s prompt, this traffic pattern is triggered, allowing the adversary to establish a covert channel that encodes the presence of specific keywords into observable traffic features. Simultaneously, we employ different watermarks to embed encodings of the user’s input prompt across multiple conversation turns, thereby increasing the information capacity of the covert channel. This attack is feasible because certain watermarks, such as “please answer briefly” and “web search”, can significantly alter traffic patterns, including packet size and inter-packet timing. These changes remain observable even under encrypted traffic settings, such as when VPN tools are in use. Critically, these watermarks (prompt constraint terms) are inherently benign. They neither exfiltrate information to remote endpoints nor execute malicious code locally. As a result, existing skill security auditing tools do not classify them as risky behavior.

A motivating example is as follows. When a grandmother consults the agent about elderly-friendly hypoglycemic foods, the attacker aims to recover three attributes: the user’s age group, the query domain, and the goal. Prior fingerprinting methods construct a dataset in advance and match the collected network traffic against it. Our method departs from this paradigm. We embed watermarks in the skill used by the target, and any prompt term that causes a traffic change directly observable by the adversary can serve as a watermark. Specifically: (i) **Age inference**. In the first conversation turn, the skill checks whether the user’s prompt references senior-related terms and, if so, adopts a concise dialogue style suitable for elderly users, which the attacker observes as a smaller traffic volume. (ii) **Domain inference**. In the second

turn, the skill checks for medical terminology and, if present, retrieves medical reference images, producing a traffic burst that encodes the medical domain. (iii) **Goal inference.** In the third turn, the skill organizes candidate conditions into severity tiers and generates outputs of proportionally different sizes, encoding the severity level in the response volume. By observing the multi-turn interaction, the attacker observes a three-bit encoding of the user’s query and can infer, via a pre-constructed dictionary, that the session involves an elderly user asking a medical question of moderate severity.

However, designing such watermarks is not trivial. In the encrypted traffic setting, the encoding dimensions available to the adversary are highly constrained. The adversary cannot directly analyze agent behavior from URLs, which increases the difficulty of observation. Moreover, designing a conversational paradigm that is intuitive and that does not raise suspicion from skill detection tools is of critical importance. If a skill cannot provide a reasonable explanation for the specific watermarks, the detection tool will question the intent of the skill and flag it as risky. Finally, the degree to which different watermarks alter traffic patterns must also be quantified. If the differences between watermarks are not sufficiently pronounced, the adversary will struggle to observe the covert information encoded by the watermarks from the traffic. Therefore, designing an effective encoding toolbox is important.

Our contributions are as follows.

- We identify a hidden attack surface that exploits the relationship between prompts and their corresponding traffic patterns. We design SkillWatermark, the first attack that leverages this surface to establish a covert channel for encoding private information.
- We quantify the significance of the impact that different watermarks exert on traffic patterns across 14 domains, enabling the adversary to embed distinguishable traffic signatures within skills. We further characterize the cross-domain robustness and cross-style stability of these watermarks.
- We apply our transformation to over one hundred skills from a public skill marketplace. Extensive experiments confirm the usability of the transformation, and all transformed skills pass the review of LLM-based risk detection tools. We discuss the limitations of the current evaluation and outline directions for quantifying end-to-end information recovery accuracy.

II. BACKGROUND AND RELATED WORK

A. LLM-based agents and Skills

LLM-based agents are intelligent systems that systematically orchestrate LLMs to accomplish production tasks. Specifically, LLM-based agents operate on a think-act-observe cycle known as the ReAct framework [14]. Compared with traditional LLMs, the pipeline of an LLM-based agent no longer relies on single-turn or multi-turn question answering; instead, it completes tasks through autonomous decision-making and internal multi-step, goal-driven execution flows. Agents have

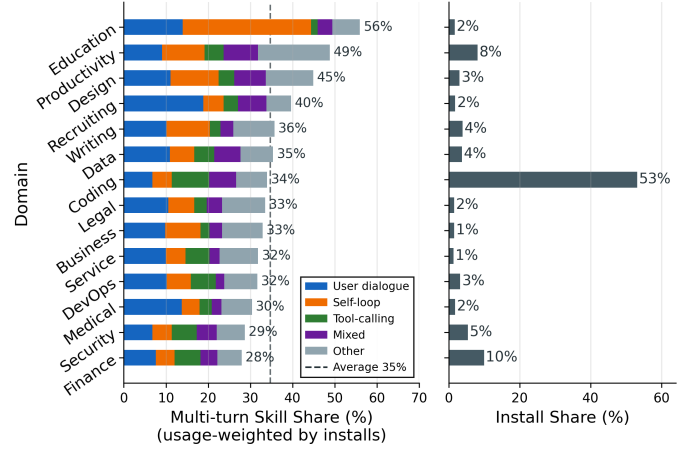


Fig. 1: Multi-turn skill prevalence across the full ClawHub skill marketplace. **Left:** per-domain share of multi-turn skills, usage-weighted by install count. **Right:** each domain’s share of total installs.

been practically deployed across numerous domains to improve productivity [4]–[7].

A skill is an extension package for agents that standardizes and guides agent behavior through a set of structured prompts and optional scripts, with the goal of improving output quality to approach domain-expert capability. A typical skill consists of a Skill.md file with corresponding YAML-formatted frontmatter. Optional scripts support processing tasks such as file reading and execution, and are commonly written in Python, shell, and JavaScript. Any user may submit customized skills to open-source registries such as OpenClaw’s ClawHub [15]. At present, mainstream production-grade agents already support this skill interface [1], [3], [16].

We crawled 65,699 skills published on ClawHub as of June 2026, and present the resulting statistics in the Figure 1. The left panel depicts the proportion of skills that possess multi-turn conversation capability, weighted by download count to account for the fact that many skills contain minimal descriptions and are rarely used. Following the previous works [17], [18], we define a skill as possessing multi-turn conversation capability when it requires multiple rounds of user inquiry or internal multi-step iteration to complete a task. Among all collected skills, those with multi-turn capability account for more than one third of the total. In education domains, this share exceeds 50 percent of the market. We further present the download distribution across domain categories to illustrate the degree of skill adoption in different areas. Over half of all skills are used for coding. Notably, other domains such as finance and productivity each account for approximately 10 percent of total downloads. The skill marketplace is currently experiencing rapid growth, which reflects the potential for skill adoption in an expanding range of domains.

B. Skills Security

Although open skill ecosystems accelerate capability growth and developer adoption, they also introduce a supply chain risk

TABLE I: Taxonomy of malicious behaviors exhibited by agent skills.

Type	Description
Data exfiltration	Transmitting sensitive information to external endpoints.
Privilege abuse	Escalating privileges beyond the stated purpose of the skill.
Remote execution	Downloading and executing code remotely.
Prompt injection	Injecting instructions to bypass the agent’s security controls.
Content manipulation	Introducing bias and false content into the output.

that has been broadly overlooked. Existing skill classification methods [19]–[22] categorize current malicious behaviors of skills. Table I summarizes these categories.

To identify malicious skills, a number of malicious skill detection tools have been proposed [19], [23], [24]. Since skills consist of two components, namely the natural language description in the Skill.md manifest and the corresponding code, current malicious skill detection tools employ two analytical approaches. The first is static analysis, which checks whether a file contains code blocks or natural language description blocks matching the categories of malicious behavior listed in Table I. The second is LLM-based analysis, which uses different LLMs to further scrutinize the natural language content of the Skill.md manifest. Beyond research prototypes, agent vendors themselves have introduced security audits and runtime protection mechanisms. Finally, security auditing tools execute these skills within sandboxed environments to check whether they transmit information to third parties or attempt privilege escalation.

In this paper, we propose a general method for transforming benign skills such that they can leak information without employing any of the established malicious skill attack paradigms. Fundamentally, our skill transformation approach consists of a composition of benign skill statements (watermarks), and therefore it does not belong to any known malicious skill category. LLM-based analysis likewise cannot classify skills transformed in this manner as malicious, as demonstrated in Section V.

C. Network Traffic Analysis of Agent

Weiss et al. [25] attempted to reconstruct prompts by inferring entire word sequences from the relationship between individual words in each prompt and the corresponding streaming token lengths, which are reflected in packet sizes. Several providers [26] have since deployed countermeasures such as traffic padding and token batching, which render [25] ineffective under current conditions. NETECHO [27] demonstrated that the reconstruction of natural language text from token sequences remains viable in certain specialized domains, including medicine and law. However, this classification approach relies on domain-specific structured vocabularies. WHISPER LEAK [28] and AGENTPRINT [9] investigated the possibility of inferring private information through passive network traffic

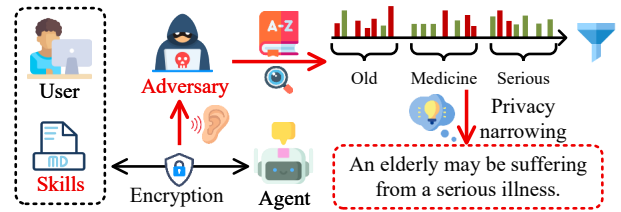


Fig. 2: The threat model of SkillWatermark.

observation and represent the work most closely related to ours. Their methods, however, adapt website fingerprinting and app fingerprinting techniques and implement attacks through a pipeline of traffic collection, model training, and classification. Since user prompts can be arbitrarily diverse, the effectiveness of these attacks depends directly on whether the prompt appears in the training dataset. A recent study [10] proposed inference based on more fine-grained traffic information, operating under the assumption that the user does not employ tools such as VPNs for encryption and performs web search activities. Under this setting, the attacker can explicitly obtain the domain names of external web pages accessed by the agent and use them to infer the user’s prompt. However, it remains dependent on the observability of Domain Name System (DNS) and Server Name Indication (SNI). Rather than collecting traffic for a pre-defined corpus of prompts and training a classifier, SkillWatermark actively shapes traffic through prompt-level watermarks to encode information into observable patterns.

III. THREAT MODEL AND MOTIVATION

A. Threat Model

We consider a local and passive adversary who attempts to infer sensitive user information from agent interactions, specifically the user’s intent, and further to deduce the user’s identity from latent attributes implied by that intent. The adversary can only collect packet traces from the connection between the client and the agent server. Similar adversarial models are well established [9], [11], [29], encompassing network administrators, token relay service providers, and internet service providers, among others. The adversary lacks decryption capability and is unable to modify, drop, delay, or decrypt any packet. In contrast to prior work that assumes direct observation of SNI, which is unavailable under encrypted traffic, we relax this assumption and permit the adversary to observe only packet size and direction, metadata that remain observable even under encryption.

The adversary can upload and promote crafted skills through supply chains such as skill marketplaces and GitHub. Our testing confirms that because these skills exhibit no overtly malicious behavior, existing auditing methods pass all modified skills without raising alerts. We assume that users employ such skills for their agent interactions. Unconscious use of skills by the user generates distinctive traffic patterns. The adversary observes these patterns, consults a pre-constructed dictionary, and thereby extracts covert information across multiple rounds

of interaction between the user and the agent. The adversary can correlate these fragments of private information and perform privacy inference to deduce the user’s input prompt. Users may adopt anonymous proxies such as Clash [30] and V2Ray [31], which reduces data observability and limits the adversary to basic metadata including packet size, direction, and timestamps. We further assume that the user initiates at least one conversation turn. The adversary can observe the traffic of a single session at a given time.

B. Motivation

The core insight of SkillWatermark is that user prompts are filled with a large volume of extraneous information, and the key to information leakage lies not in fully matching the entire prompt, for instance by collecting traffic and training a classification model. The genuinely private information is instead concealed within only a few words.

We illustrate this observation with a compelling example. A user poses two distinct questions to an agent, each inquiring about a different astronomical concept, namely planets and black holes. We repeatedly collected 30 sessions for each question and visualized the resulting traffic using t-SNE. Furthermore, we enlisted an LLM as a language expert to rephrase the questions using diverse conversational styles, thereby simulating the linguistic habits of different user populations in real-world settings. As shown in Figure 3, once the user’s language style changes, the box plot of traffic distribution deviates substantially from the baseline. Notably, across different linguistic styles we observed differences in traffic volume exceeding a factor of ten. The t-SNE visualizations illustrate this insight even more directly. When the two astronomy questions are posed using a fixed conversational style, the t-SNE projections of the two question categories exhibit a clear separation boundary. When the questions are rephrased using diverse conversational styles, however, the distributions of the two categories in the t-SNE projection overlap completely.

This example motivates a critical realization. If the adversary’s objective is to leak the key information embedded in the traffic, such as the concepts of planets and black holes in the example above, exhaustively collecting all possible sessions is impractical. In principle, prompts containing specific sensitive vocabulary can be personalized by users into arbitrarily diverse patterns. Our key observation is that the adversary need only maintain a dictionary of terms of interest and encode the corresponding prompt features into structured traffic patterns to leak the covert information.

IV. DESIGN OF SKILLWATERMARK

A. Overview

As outlined in our threat model, the adversary’s goal is to ascertain the user’s private information through traffic observation. Prior approaches that collect traffic for a large corpus of prompts and then train classification models suffer from fundamental impracticality in real-world privacy inference, because the adversary cannot guarantee that the user’s prompt

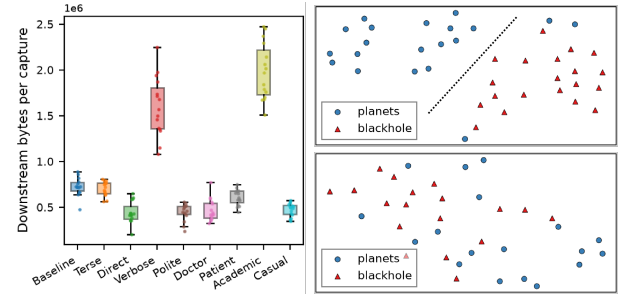


Fig. 3: (a) Incoming traffic volume for a single intent (diabetes treatment) rephrased in nine linguistic styles. (b) t-SNE of traffic features for two domains, namely *planets* and *black-hole*. Up: fixed prompt description. Down: style prompt description.

actually belongs to the collected dataset. Our key observation is that the private information the adversary seeks to acquire, such as the critical terms within a prompt, may involve only a handful of words. For example, when a medical graduate student uses a coding skill to write code related to tumor diagnosis, prior traffic-based privacy inference would classify this user as a programmer rather than a medical student. If the adversary can detect the term tumor diagnosis through traffic observation, however, the user’s professional identity has already been disclosed. The objective of SkillWatermark is therefore to design watermarks, a set of specially crafted prompt terms, that shape traffic to facilitate observation, so that information is unconsciously leaked across multiple agent interaction turns. However, this is a non-trivial undertaking, as the adversary faces three challenges in this setting:

- **Packet encryption.** The adversary cannot observe any finer-grained information, such as the destination IP address of web search requests. Prior work has used such information to construct user profiles. Furthermore, under this setting, the adversary cannot directly determine whether a given user interaction involves a skill invocation, web browsing, or other types of web activity.
- **Security auditing.** Skill publication and distribution may undergo review by security auditing tools. Although our traffic encoding does not appear in any known static skill screening list, LLM-based security auditing tools may analyze the skill description and identify potentially malicious behavior. The adversary must ensure that the traffic encoding does not trigger alerts from these security auditing tools.
- **Information density.** The adversary must embed as much information as possible within each conversation turn when designing the skill, so as to complete information leakage within the fewest possible turns. Excessive turns may arouse the user’s suspicion.

This paper presents SkillWatermark, a novel framework for designing embedded skill watermarks that enable adversaries to progressively reveal private information across multiple rounds of traffic observation. The framework addresses the challenges of information leakage in encrypted traffic environments by leveraging linguistic description templates and

a multi-level encoding strategy. First, we design a general-purpose encoding toolbox that can encode arbitrary information into features observable by the adversary at the traffic level through several watermarks. This observation does not rely on any form of traffic decryption and remains effective even when the user employs anonymity tools such as VPNs. Second, we design a set of linguistic description templates that allow the adversary to embed these encodings into natural language without raising suspicion from LLM-based security auditing methods. Third, we propose a multi-level encoding strategy that allows the adversary to progressively narrow the scope of the content described in the user’s prompt across multiple conversation turns to increase the amount of information that can be transmitted within a single prompt input.

B. Watermark Encoding Toolbox

We first consider the metadata available in encrypted traffic, which determines the encoding dimensions accessible to the adversary. Under traffic encryption, most information is concealed, leaving the adversary with only three observable dimensions, namely timing, direction, and packet size. We consider each encoding strategy in turn.

- **Timing.** The adversary can obtain the timestamp of each burst. We define a burst as the traffic surge observable during each conversation turn. We normalize these timestamps as offsets relative to the first packet. In multi-turn conversations, a skill can deliberately set time watermarks to impose spacing between turns.
- **Direction.** The adversary can observe outgoing traffic, defined as user-to-agent packets, and incoming traffic, defined as agent-to-user packets. Using direction information, the adversary can determine the turn boundaries of a conversation. Multi-turn conversations further allow watermarks to be applied multiple times, thereby increasing the amount of information that can be leaked. That is, a single user input is decomposed by SkillWatermark into multiple conversation turns, and the traffic pattern of each turn can be observed.
- **Packet size.** The adversary can observe the size of each individual packet. In multi-turn conversations, a skill can enforce a specific response volume by setting a watermark. The adversary can influence and control the specific size of each outgoing and incoming burst through watermarks. The watermarks restrict the number of incoming characters output by the agent, thereby controlling the size of incoming bursts.

Watermark description. For the adversary, the most direct approach to modifying the aforementioned dimensions is to write scripts that control the interaction between the agent and the LLM, for instance by embedding pad and wait commands through hooks [32]. However, this requires the adversary to additionally provide corresponding executable scripts with the skill. We aim to design a watermark paradigm that relies solely on prompt descriptions, so that the watermark can be applied to any skill rather than only those that ship with scripts. Therefore, we need to identify prompt terms that can induce significant traffic changes to serve as watermarks. Our

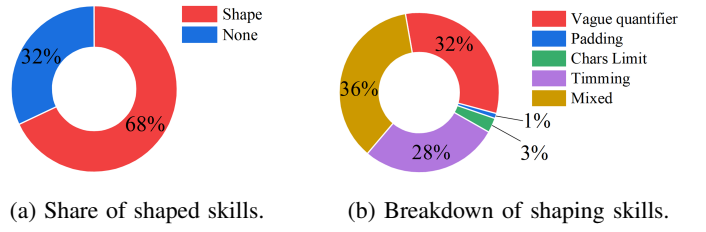


Fig. 4: Prevalence of traffic-shaping behaviors across collected ClawHub skills. (a) Share of total *installs* accounted for by skills whose behavior shapes traffic. (b) Breakdown of the shaping skills into mutually exclusive categories.

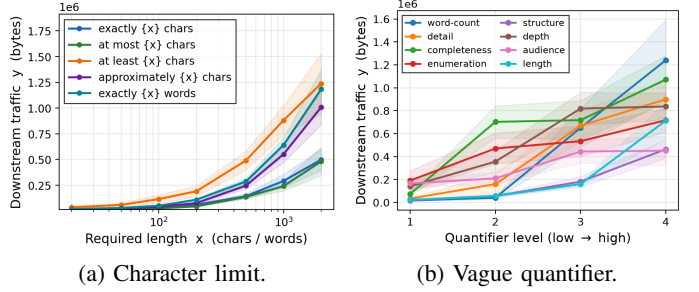


Fig. 5: Incoming traffic vs prompt length-control across different prompts. Left: character limit. Right: vague quantifier.

insight is that existing benign skills already contain structurally similar descriptive patterns. As shown in Figure 4, we conducted a statistical analysis of the proportion and distribution of skills on ClawHub that inadvertently shape traffic through their descriptions and scripts. Approximately 68 percent of skills installed by users employ such descriptions, thereby exerting a potential influence on traffic patterns. Therefore, we search for analogous descriptions within benign skills and splice them together to form the watermarks. This is the key design principle that enables our offensive infrastructure to evade detection, namely that our traffic-level adjustments are fundamentally a composition of output patterns drawn from benign skills.

Packet size. We first consider the packet size dimension. The most direct approach to controlling character output is to embed watermarks that explicitly specify the output length, such as “at least x characters” or “no more than y words”. However, the encoding introduced by such explicit character limits may appear suspicious to LLM-based reviewers. We therefore identify a set of more vague descriptions from existing skills, such as “as few as possible” or “more than a few”. We refer to these descriptions as *character limit* and *vague quantifier*, respectively. We separately evaluate the impact of these two types of prompt constraint terms on packet size.

Figure 5 illustrates the degree of control that different watermark terms exert over the agent’s incoming traffic. We find that among character limit watermarks, “at least x chars” performs the best and produces the most discriminative in-

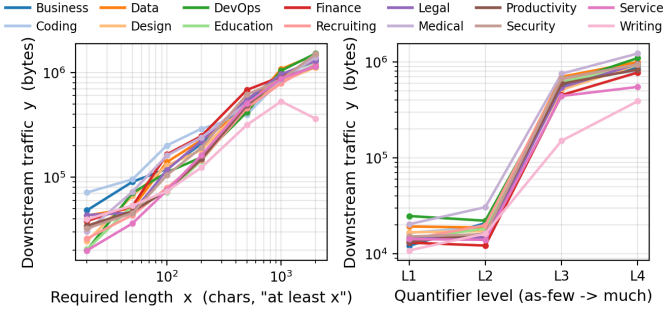


Fig. 6: Incoming traffic vs prompt length-control across 14 domains. Left: “at least x characters”. Right: vague quantifier (4 levels, as-few, fewer, more, much).

coming traffic patterns. In contrast, “exactly x chars” and “at most x chars” perform considerably worse. We hypothesize that this is attributable to the generative mechanism of LLMs. For vague quantifiers, the level of detail provides a constraint on incoming traffic volume that most closely approximates an explicit character limit. We therefore use the level of detail to conceal the encoding of specific character count constraints (i.e., “as-few/few/more/much chars”). We further evaluated the effectiveness of these two types of watermark across 14 different domains and summarize the results in Figure 6. We find that both watermarks exhibit the strongest incoming traffic control capability across all domains, with relatively low cross-domain fluctuation in traffic volume. This indicates that encoding traffic volume using these watermarks possesses substantial cross-domain robustness, which is critical for enabling the adversary to encode domain-specific user profiles through packet size.

Timing. We design concealment strategies for the timing encoding behavior. As noted above, the adversary can directly design prompt constraint terms for the skill, such as instructing the agent to delay its response by x seconds, or execute a local task with a specific time complexity, for instance by sleeping for a defined interval. We find, however, that the response delays introduced by such prompt constraints conflict with the packet size dimension. Neither explicit instructions such as “wait for x seconds” nor their concealed variants such as “think carefully before answering” can produce a incoming traffic delay without side effects. The additional waiting process still generates tokens related to the reasoning phase, which causes the burst size of a single response to change and thereby introduces a cross-dimension conflict. Under encrypted traffic settings, the adversary cannot distinguish the reasoning phase from the actual answer phase solely from traffic patterns.

We explore increasing the agent’s response delay by raising computational complexity. However, embedding excessively high computational complexity in a watermark would be flagged as malicious consumption of system resources. The watermarks we design must therefore be semantically reasonable and of the lowest possible computational complexity. A straightforward approach to designing timing watermarks is to

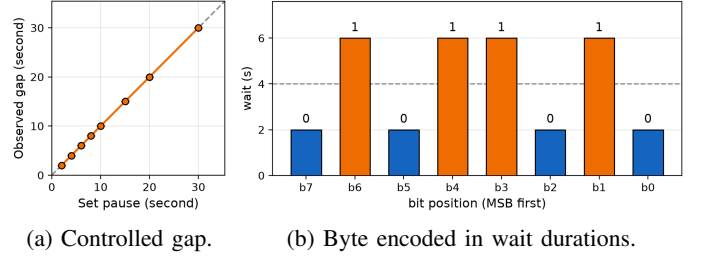


Fig. 7: Timing encoder capacity of a wait-tool skill. (a) A skill injects a silence of exactly N seconds between the two incoming bursts. (b) The controlled timing sequence doubles as a covert timing channel.

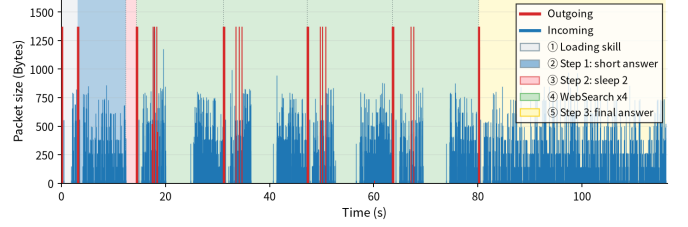


Fig. 8: An example of an embedded timing watermark, with one Bash sleep of 2 seconds and four web searches inserted before the final answer.

have the skill execute `sleep` through bash, thereby inserting specific encoding gaps between conversation turns. Such commands introduce no additional computational complexity because they consume no system resources whatsoever. Figure 7a shows the relationship between the designed time watermarks and the time interval that the adversary actually observes between conversation turns. Time watermarks designed in this manner produce strictly defined time intervals in the traffic. Therefore, the adversary can control the covert information transmission by controlling the time interval between each turn, as illustrated in Figure 7b.

We are similarly concerned about the potential audit risks that such descriptions may introduce, particularly from LLM-based reviewers. We therefore select tasks from existing skills that introduce additional computational complexity to serve as concealed watermarks. The framework encodes whether the agent performs a web search operation during each conversation turn, thereby introducing a controllable delay in each turn. Figure 8 illustrates a comparison between the traffic behavior of web search actions and direct answer actions (step 1 and step 3), with the web search performing four rounds of retrieval. We observe that the traffic pattern during web search differs substantially from that of direct answering. In particular, the web search introduces additional waiting time because the agent performs search queries and summarizes the results in the background. Therefore, the adversary can distinguish web search actions from direct answer actions through direct traffic observation. Given that different models adopt different search strategies, we collected data on four distinct agents.

This confirms that web search behavior constitutes a traffic pattern with a characteristic latency signature and can serve as a prominent timing watermark.

C. Multi-Level Encoding Strategy

We next consider how to structure the information to be transmitted so that (1) the adversary can locate when a user employs this skill from the traffic, and (2) the adversary can leak the information of interest through the traffic. We achieve the above goals by configuring watermarks as distinguishable multi-turn conversations. Inserting a segment of natural language description into a skill does not directly produce a multi-turn conversation, because the agent sends the prompt and the skill description to the LLM together rather than turn by turn. Generating a multi-turn conversation during agent-LLM interaction therefore requires following a specific paradigm. We summarize the paradigms that can be used for multi-turn watermark design as follows.

- 1) Orchestrating tool-call sequences using natural language. Within a single-turn design, a specific tool is used to block the current session. This tool can be designed to run locally, such as locally executed scripts or commands, or remotely, such as invoking web search or other agent tools. These tool calls break the original single-turn conversation into multiple turns, thereby creating sufficient encoding space for the adversary.
- 2) Inserting reasoning between calls. Natural language descriptions must be inserted between multi-turn segments to form a semantically coherent logical chain, preventing the appearance of isolated and suspicious descriptions. For instance, the multi-turn structure can be semantically framed as common logical processes such as reflection and revision, plan and execution, or candidate proposal and evaluation. This ensures that the natural language descriptions embedded in the watermark are naturally consistent with the surrounding context. Such multi-turn paradigms are in fact already widely adopted in existing skills. Our watermark design paradigm therefore minimizes the likelihood of arousing suspicion from LLM-based security analyzers.
- 3) Embedding specific covert information into the available encoding space. We formalize this problem as follows. The adversary designs an N -round conversation procedure, where N equals the length of P . P is defined as a binary sequence. When the i -th element is 0, the skill initiates a prompt of type A. When the i -th element is 1, the skill initiates a prompt of type B. Here A and B represent two distinct traffic pattern pairs triggered by two different watermarks. The adversary therefore configures the outgoing sequences of the N rounds as a specific trigger, while simultaneously encoding covert information in the timing and incoming packet size dimensions, thereby creating sufficient encoding space across N rounds to transmit private information.

Through the above design strategies, the adversary can achieve two goals. First, the adversary can design a unique

traffic pattern that is distinguishable from any pattern in the existing skill marketplace, and determine whether a user is using a specific skill by observing whether the corresponding watermark traffic is generated. Second, the adversary maintains multiple groups of objects of interest and encodes them into the skill using different watermarks to determine whether the user's prompt contains content of interest to the adversary. We illustrate the encoding scheme with the following instantiation. Suppose the adversary maintains a dictionary of K sensitive keywords $\{w_1, \dots, w_K\}$. For each keyword w_k , the adversary assigns a distinct binary codeword of length N (the number of conversation turns). In each turn $i \in \{1, \dots, N\}$, the skill checks whether the user's prompt contains any keyword whose i -th codeword bit is 1. If so, the skill executes watermark pattern A (e.g., a brief answer followed by `sleep`); if not, the skill executes watermark pattern B (e.g., a detailed answer with web search). The adversary observes the traffic across all N turns, classifies each turn as pattern A or B, and concatenates the bits to recover which keyword was present. With $N = 4$ turns and two distinguishable patterns per turn, the adversary can discriminate among up to $2^4 = 16$ keywords in a single session. We evaluate the effectiveness of SkillWatermark in Section V.

V. EVALUATION

A. Experiment Setup

Dataset. We crawled 65,699 skills from ClawHub and categorized them by domain into 14 specialized domains. Because different skills employ distinct and highly specialized prompts, we simplify the prompt design by selecting the 10 most-downloaded skills in each domain for testing. For each skill we design questions that match its intended functionality, embedding identity, goal, and requirement information within them to simulate prompts that are as realistic as possible. To preserve the original functionality of each skill, we invoke the watermark independently at the beginning of the original skill. In total, we captured over 7,000 samples for evaluation.

Implementation. To facilitate data collection, we designed an automated collection script following the approach of Whisper Leak [28]. We installed Claude Code [1] on a laptop equipped with an Intel i7-13700H CPU and configured the runtime environment. During data collection, a Clash proxy was used to simulate the scenario in which a user employs a VPN. For ease of evaluation, we designed a three-turn conversational watermark and label each stage as T1 through T3.

B. Traffic Consistency

We first evaluate the consistency between the traffic generated by skills embedding a watermark and that generated by ordinary skills. If the traffic for specific questions is highly consistent, the adversary can confirm the user's intent through direct observation, without the need to repeatedly collect data and train models. This is the core distinction between our method and prior work.

We report consistency using two types of metrics, namely statistical metrics based on Wasserstein distance [33] and

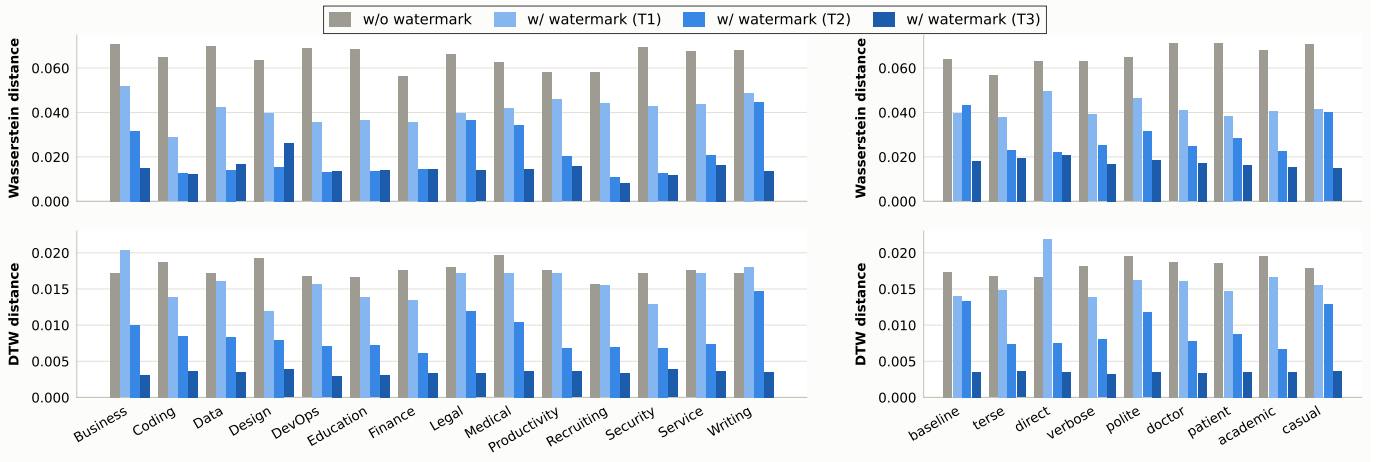


Fig. 9: Comparison of Wasserstein distance and DTW distance between original skills and skills with an embedded three-stage watermark. Smaller distances indicate better intra-class consistency. Left: cross-domain consistency evaluation. Right: cross-prompt-style consistency evaluation.

packet-level metrics based on Dynamic Time Warping (DTW) distance [34]. The lower the metric, the smaller the difference between traffic captures from different sessions, indicating more stable traffic. First, to evaluate generalizability, we assess whether the designed watermark is effective across skills from different domains. The left subfigure of Figure 9 presents the effect of embedding the watermark into skills from 14 different domains. We observe that after embedding the watermark, traffic consistency improves markedly, with gains in both statistical features and packet-level consistency. In particular, by the second conversation turn, 11 domains already exhibit more than a twofold improvement in consistency compared to the unwatermarked baseline. By the third turn, the traffic patterns of different skills exhibit highly similar structures. This indicates that the embedded watermark substantially enhances the observability of traffic patterns, and that these specific traffic patterns reflect the intent of the skill being used by the user.

Next, we examine whether our watermark can detect the corresponding traffic patterns when user prompts are topically related but stylistically diverse. This simulates the impact of different users’ individual habits on the traffic patterns produced by prompts. We use the nine linguistic styles listed in Table II. The right subfigure of Figure 9 illustrates the impact of various prompt styles on traffic. We also observe that after the traffic is shaped by the watermark, the degree of structure increases while randomness decreases. We note that the DTW distance still exhibits significant randomness during the first watermark turn, which is caused by a conflict between the prompt style and the first-turn watermark. For instance, the Academic style demands more complex answers, whereas the first turn only requires a brief response. As the turn count increases, however, the degree of structuring also rises. By the third turn, the conversation already exhibits a high degree of consistency. This demonstrates that the designed watermark maintains stability across prompts of different

TABLE II: Prompt style

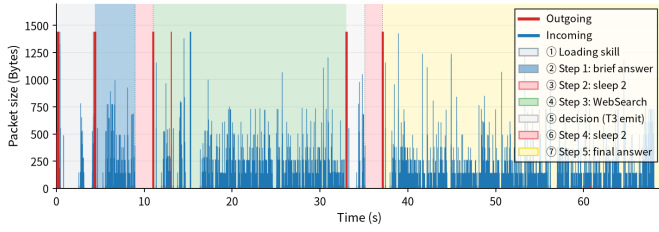
Style	Example
Baseline	Give me some suggestions.
Terse	Suggestions?
Direct	Just tell me what to do.
Academic	I would appreciate receiving evidence-based recommendations pertaining to this matter.
Casual	Hey, any tips for me?
Verbose	I was wondering if you might be able to provide me with a comprehensive set of suggestions that could potentially help me address this situation in the most effective manner possible.
Polite	Could you kindly offer some suggestions?
Doctor	Based on a profession’s perspective, what management strategies would you recommend?
Patient	Can you help me understand what I should do about this?

styles but identical intent, and can effectively encode the user’s intent into traffic patterns that are readily observable.

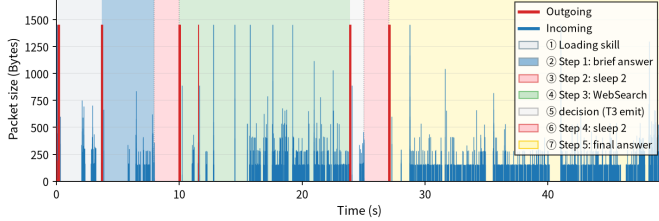
Next, we evaluate the watermark’s effectiveness across different LLMs. We add GLM 5.2 for testing and present the traffic patterns of the sample watermark under different models. Figure 10 presents the watermark shapes under different models. Although different models exhibit differences in fine-grained packet distributions, for instance the traffic of GLM is sparser, the behavior of each conversation turn encoded by the watermark exhibits a high degree of consistency across multi-turn interactions. This is precisely the key distinction between our watermark scheme and prior traffic collection approaches, namely that we need not collect large volumes of data under many influencing factors. The adversary need only observe the corresponding behavioral pattern to confirm that the user has triggered a specific watermark.

C. Decoding Evaluation

Next, we evaluate whether the implicit information encoded in the watermarks can be recovered from the traffic patterns in



(a) Watermark traffic pattern when using Claude Code.



(b) Watermark traffic pattern when using GLM 5.2.

Fig. 10: Cross-model watermark consistency evaluation. The model API was switched by modifying the settings.local.json file, and traffic was then collected.

TABLE III: Evaluation of TPR and FPR on datasets with and without embedded watermarks.

Criterion	TPR	FPR
watermark (T2)	92.5%	13%
watermark (T2+T3)	98.8%	8%

the collected samples. We assess the True Positive Rate (TPR) and False Positive Rate (FPR) by testing on traffic traces with embedded watermarks and on traffic traces without embedded watermarks. The results are presented in the table. Since the consistency evaluation indicates that T1 provides limited discriminability for traffic, we use T2 and T3 as the watermark identification indicators. As the number of conversation turns increases, TPR rises while FPR declines. On the watermarked dataset, our method achieves a decoding success rate of 98.8 percent. On the unwatermarked dataset, the false positive rate is only 8 percent.

D. Anti-detection Evaluation

resume-optimizer

...

If the user provides only a Job Description without a resume, prompt for resume materials.

If the user provides only a resume without a Job Description, proceed in `rewrite` mode and prompt after Step 2...

...

Finally, we evaluate whether our attack method can pass existing detection tools. Following the previous works, we conduct a qualitative study applying LLM-based detection

tools to assess the stealthiness of our attack. We find that even when LLM auditing tools explicitly observe the watermark content, they do not classify the watermark as belonging to any known attack category. In particular, when the watermark attempts to leak specific implicit information, for instance by using pattern A for elderly users and pattern B for younger users, the LLM audit naturally interprets this as personalization intended to better serve specific user populations. It does not regard such targeted analysis as a privacy inference risk, because many legitimate skills employ similar strategies. We list below an example from existing publicly available skills that already employ mechanisms similar in structure to our watermarks [35]. The adversary need only observe whether the corresponding traffic is generated to determine whether the user provided a Job Description or a resume. This illustrates the weakness of existing detection paradigms: traffic side-channel leakage does not require any of the known malicious-skill primitives, and current auditing frameworks are not designed to detect it.

VI. DISCUSSION AND CONCLUSION

Countermeasures. Our attack is grounded in a fundamental observation that specific prompts of skills generate distinguishable traffic patterns, which are inadvertently encoded as skills optimize for superior output performance. These interaction patterns establish an implicit side channel within network traffic that an attacker can exploit to compromise user privacy. The most straightforward countermeasure would be to prohibit any description for producing structured traffic patterns during multi-turn conversations. However, this approach runs directly counter to the foundational purpose of skill design, as structured multi-turn interactions are intended to enhance task completion efficiency and to reduce the frequency of manual user intervention. We therefore advocate that existing skill auditing frameworks incorporate an additional detection dimension that specifically inspects skills capable of generating patterned traffic across conversation turns, thereby strengthening the community’s defense perimeter.

Limitations. Our attack paradigm is applicable only to LLMs whose network traffic can be observed. If a user deploys the model in an offline setting, our attack does not take effect. Scenarios in which the user does not employ any skill likewise fall outside the scope of this work. If a user conducts only a single conversation turn with a very short prompt, the amount of leaked information remains limited. When a user initiates multiple concurrent sessions, we are unable to isolate individual sessions because the VPN masks port numbers, and the method therefore does not apply. False positives can arise when unmodified skills coincidentally produce traffic resembling the watermark. The adversary mitigates this by avoiding watermark designs that replicate existing skill descriptions.

Conclusion. In this paper, we proposed SkillWatermark, a novel attack paradigm that exploits the embedding of specific prompt terms to generate distinctive traffic patterns. These watermarks are benign structured statements, and as a result existing detection paradigms overlook them. Our experiments

demonstrate that watermarked skills produce highly consistent traffic patterns across 14 domains and diverse prompt styles, and that LLM-based auditing tools do not flag the transformed skills as malicious. This paper presents the first systematic exposition of how covert channels can be constructed in agent skills using prompt-level watermarks, and reveals the blind spot of existing skill security auditing tools with respect to traffic side-channel leakage.

REFERENCES

- [1] A. PBC. (2026) Claude code. [Online]. Available: <https://claude.com/product/claude-code>
- [2] I. Anysphere. (2026) Cursor. [Online]. Available: <https://cursor.com>
- [3] OpenAI. (2026) Codex. [Online]. Available: <https://github.com/openai/codex>
- [4] W. Wang, Z. Ma, Z. Wang, C. Wu, J. Ji, W. Chen, X. Li, and Y. Yuan, “A survey of llm-based agents in medicine: How far are we from baymax?” in *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, ser. Findings of ACL, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds., vol. ACL 2025. Association for Computational Linguistics, 2025, pp. 10345–10359. [Online]. Available: <https://doi.org/10.18653/v1/2025.findings-acl.539>
- [5] Y. Yu, H. Li, Z. Chen, Y. Jiang, Y. Li, J. W. Suchow, D. Zhang, and K. Khashanah, “Finmem: A performance-enhanced LLM trading agent with layered memory and character design,” *IEEE Trans. Big Data*, vol. 11, no. 6, pp. 3443–3459, 2025. [Online]. Available: <https://doi.org/10.1109/TBDATA.2025.3593370>
- [6] Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P. S. Yu, and Q. Wen, “LLM agents for education: Advances and applications,” in *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Association for Computational Linguistics, 2025, pp. 13782–13810.
- [7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper/_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [8] R. Xu and Y. Yan, “Agent skills for large language models: Architecture, acquisition, security, and the path forward,” *CoRR*, vol. abs/2602.12430, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.12430>
- [9] Y. Zhang, X. Deng, Z. Gu, Y. Chen, K. Xu, Q. Li, and J. Wu, “Exposing LLM user privacy via traffic fingerprint analysis: A study of privacy risks in LLM agent interactions,” *CoRR*, vol. abs/2510.07176, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.07176>
- [10] H. Jeong, M. Teymorianfard, A. Kumar, A. Houmansadr, and E. Bagdasarian, “Network-level prompt and trait leakage in local research agents,” *CoRR*, vol. abs/2508.20282, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2508.20282>
- [11] J. Li, H. Zhou, S. Wu, X. Luo, T. Wang, X. Zhan, and X. Ma, “FOAP: Fine-Grained Open-World android app fingerprinting,” in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 1579–1596. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/li-jianfeng>
- [12] J. Gong, G. Meng, W. Cai, S. Liang, T. Wang, and E.-C. Chang, “Foada: Toward robust open-world mobile app fingerprinting,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 8014–8029, 2025.
- [13] X. Deng, R. Zhao, Y. Wang, M. Zhan, Z. Xue, and Y. Wang, “Countmamba: A generalized website fingerprinting attack via coarse-grained representation and fine-grained prediction,” in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 1419–1437.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [15] OpenClaw. (2026) Clawhub. [Online]. Available: <https://clawhub.ai/>
- [16] Microsoft, “Microsoft agent framework,” 2026. [Online]. Available: <https://learn.microsoft.com/en-us/agent-framework>
- [17] Y. Zhou, W. Shu, Y. Su, W. Du, Y. Fang, and X. Lin, “A comprehensive survey on agent skills: Taxonomy, techniques, and applications,” *CoRR*, vol. abs/2605.07358, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2605.07358>
- [18] Y. Li, X. Shen, Y. Miao, X. Yao, X. Ding, R. Krishnan, and R. Padman, “Beyond single-turn: A survey on multi-turn interactions with large language models,” *Trans. Mach. Learn. Res.*, vol. 2026, 2026. [Online]. Available: <https://openreview.net/forum?id=UYNQXPevpF>
- [19] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, ““do not mention this to the user”: Detecting and understanding malicious agent skills in the wild,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.06547>
- [20] Z. Li, J. Wu, X. Ling, X. Cui, and T. Luo, “Towards secure agent skills: Architecture, threat taxonomy, and security analysis,” *CoRR*, vol. abs/2604.02837, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2604.02837>
- [21] D. Schmotz, L. Beurer-Kellner, S. Abdelnabi, and M. Andriushchenko, “Skill-inject: Measuring agent vulnerability to skill file attacks,” *arXiv preprint arXiv:2602.20156*, 2026.
- [22] W. Guo, W. Zeng, C. Liu, X. Jia, Y. Xu, L. Tang, Y. Fang, and Y. Liu, “Malskillbench: A runtime-verified benchmark of malicious agent skills,” *arXiv preprint arXiv:2606.07131*, 2026.
- [23] Y. Liu, W. Wang, R. Feng, Y. Zhang, G. Xu, G. Deng, Y. Li, and L. Zhang, “Agent skills in the wild: An empirical study of security vulnerabilities at scale,” *CoRR*, vol. abs/2601.10338, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2601.10338>
- [24] Y. Hou and Z. Yang, “Skillsieve: A hierarchical triage framework for detecting malicious AI agent skills,” *CoRR*, vol. abs/2604.06550, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2604.06550>
- [25] R. Weiss, D. Ayzenshteyn, and Y. Mirsky, “What was your prompt? a remote keylogging attack on AI assistants,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3367–3384. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/weiss>
- [26] Cloudflare. (2026) Mitigating a token-length side-channel attack in our ai products. [Online]. Available: <https://blog.cloudflare.com/ai-side-channel-attack-mitigated/>
- [27] Z. Zhang, G. Wu, S. Deng, S. Wang, and Y. Zhang, “Netecho: From real-world streaming side-channels to full LLM conversation recovery,” *CoRR*, vol. abs/2510.25472, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.25472>
- [28] G. McDonald and J. B. Or, “Whisper leak: a side-channel attack on large language models,” *CoRR*, vol. abs/2511.03675, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.03675>
- [29] X. Deng, Q. Li, and K. Xu, “Robust and reliable early-stage website fingerprinting attacks via spatial-temporal distribution analysis,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1997–2011. [Online]. Available: <https://doi.org/10.1145/3658644.3670272>
- [30] MetaCubeX and Clash Contributors, “Clash meta,” <https://github.com/MetaCubeX/mihomo>, 2026, advanced version of Clash (formerly Clash Meta) with enhanced features.
- [31] v2fly and V2Ray Contributors, “V2ray: A platform for building proxy applications,” <https://github.com/v2fly/v2ray-core>, 2026, core proxy platform supporting VMess, VLESS, Trojan, Shadowsocks and more.
- [32] Anthropic, “Hooks reference,” <https://code.claude.com/docs/en/hooks>, 2026, official documentation on lifecycle hooks and Agent Skills.
- [33] V. M. Panaretos and Y. Zemel, “Statistical aspects of wasserstein distances,” *Annual Review of Statistics and Its Application*, vol. 6, no. 1, p. 405–431, Mar. 2019. [Online]. Available: <http://dx.doi.org/10.1146/annurev-statistics-030718-104938>
- [34] H. Sakoe and S. Chiba, “Dynamic programming algorithm optimization for spoken word recognition,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.
- [35] wyh0626, “resume-optimizer,” <https://github.com/wyh0626/resume-optimizer>, 2026.